

```
=====
Python Keywords
=====
```

=> Keywords are reserved words in Python. These words have special meaning in Python.

=> Keywords are predefined reserved words in Python that are used to define the syntax and structure of Python programs.

=> We cannot use keywords as variable names, function names, class names, or identifiers.

// below are the reserved words in python

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue',
'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in',
'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with',
'yield']
```

this program is used to demonstrate on python keywords

```
import keyword

print(keyword.kwlist)

print(len(keyword.kwlist))

print(keyword.iskeyword("hi")) // false

print(keyword.iskeyword("import")) // true
```

```
=====
Python Literals
=====
```

=> A literal is a fixed value written directly in the program.

=> A literal is a constant value assigned to a variable or used directly in code.

```
10
10.5
"Python"
True
False
None
[10, 20, 30]
(10, 20, 30)
{"name": "Ravi"}
```

Types of Literals in Python

- 1) Numeric Literals
- 2) String Literals
- 3) Boolean Literals
- 4) None Literals

5) Collection Literals (List, Tuple, Set, Dict)

```
# integer literals
age = 25
student_id = 101

print(age)
print(student_id)

# floating literals
price = 99.99
percentage = 15.55
print(price)
print(percentage)

# String literals
name = 'Ashok'
city = "Hyd"
course = '''It's Python course'''
msg = """
    Welcome to Python Training
    with Realtime projects
    """

# Boolean Literals
is_activated = True
is_completed = False
print(is_activated)
print(is_completed)

# None Literal
result = None
print(result)

# List literal (mutable)
students = ["Ashok", "Ramesh", "Raju", "Ashok"]
print(students)

# Tuple Literal (immutable)
courses = ("Python", "Java", "C++", "Python")
print(courses)

# Set literal (no duplicates, no order)
cities = {"Hyd", "Pune", "Chennai", "Delhi", "Hyd"}
print(cities)

# Dictionary literal
student = {
    "id" : 101,
    "name" : "Ashok",
    "age" : 35,
    "gender" : "Male",
}
print(student)

students = [
    {
        "id": 101,
        "name": "Ashok",
        "age": 35,
        "gender": "Male",
```

```
    },  
    {  
        "id": 102,  
        "name": "John",  
        "age": 25,  
        "gender": "Male",  
    }  
]  
print(students)
```

```
=====  
Python Variables  
=====
```

=> A variable is a name used to store data.

=> A variable is a memory reference name that stores a value.

Ex:

```
a = 20  
b = 19.5  
name = "Ashok"
```

Note: In Python, we do not need to declare variable type.

=> In Python, variable type is decided automatically based on its value.

```
# Variables demo  
x = 10  
print(x)  
print(type(x))
```

```
name = "Ashok"  
print(name)  
print(type(name))
```

Rules for Variable Names in Python

** Variable name can contain letters, numbers and underscore(_)

** Variable name shouldn't start with number

** Variable name shouldn't contain space

** Variable name should not use special symbols

** Variable name shouldn't be reserved keyword

** Variable names are case-sensitive

```
# Variables demo  
x = 10  
print(x)  
print(type(x))
```

```
name = "Ashok"  
print(name)  
print(type(name))
```

```
_mobile = 68686868
id1 = 101
stud1 = "Rajesh"
course_fee = 4000

# assigning same value to multiple variables
var_1 = var_2 = var_3 = 100

print("Variable 1:", var_1)
print("Variable 2:", var_2)
print("Variable 3:", var_3)

# assigning different values to multiple variables in oneline
var_8, var_9, var_10 = 80, 90, 100

print("Variable 8:", var_8)
print("Variable 9:", var_9)
print("Variable 10:", var_10)

# Variable swapping
a = 10
b = 20
a,b = b,a
print(a)
print(b)

=====
Python Data Types
=====

=> A data type defines the type of value stored in a variable.

=> Data type tells Python what kind of data a variable is storing.

-----

import sys

# memory info
my_age = 30
print(my_age, type(my_age))
print("size of my_age var in bytes: ", sys.getsizeof(my_age))

my_name = "Ashok Bollepalli"
print(my_name, type(my_name))
print("size of my_name var bytes: ", sys.getsizeof(my_name))

my_age = 30
print(my_age, type(my_age))
print("size of my_age var in bytes: ", sys.getsizeof(my_age))

my_name = "Ashok Bollepalli"
print(my_name, type(my_name))
print("size of my_name var bytes: ", sys.getsizeof(my_name))

# Flot Data Type
price = 99.99
percentage = 85.5

print(price, type(price))
```

```
print("size of price : ", sys.getsizeof(price))
print(percentage, type(percentage))
print("size of percentage : ", sys.getsizeof(percentage))

# complex data type
num = 5 + 3j
print(type(num))

# list data type (ordered + mutable + duplicates allowed)
students = ["Anil", "Sunil", "Gopi", "Anil"]
print(students)
print(type(students))

# tuple data type (ordered + Immutable + Duplicates allowed)
courses = ("Java", "Python", "GEN AI", "Python")
print(courses)
print(type(courses))

# set data type (Un Ordered + Mutable + No Duplicates)
skills = {"Java", "Python", "GEN AI", "DevOps", "Cloud", "Java"}
print(skills)
print(type(skills))

# dictionary data type (key-value pair)
student = {
    "id" : 101,
    "name" : "Ashok",
    "gender" : "Male"
}
print(student)
print(type(student))

# Student Data
student_id = 101
student_name = "John"
course = "Python"
fee = 5000.00
is_paid = True
imp_concepts = ["Fundamentals", "Variables", "DSA", "OOPS"]

=====
Reading Input From User
=====

# input() is a built-in python function used to take data from the keyboard during program
execution

---

name = input("Enter Your Name : ")
age = input("Enter Your age : ")
salary = input("Enter Your Salary : ")

print("My Name : ", name)
print("My Age : ", age)
print("My Salary : ", salary)

a,b = input("Enter two numbers : ").split(" ")
print("My First Number : ", a)
```

```
print("My First Number : ", b)
```

```
---
```

```
a = "Ashok"
b = "IT"
```

```
a + b ==> AshokIT
```

```
i = 10
j = 20
```

```
i + j => 30
```

```
a = input("First Number") // str
b = input("Second number") // str
```

```
a + b => 1020
```

```
=====
Type Casting
=====
```

=> Typecasting means converting one data type into another data type.

=> Typecasting is the process of converting a value from one data type to another data type.

Why Do We Need Typecasting?

** In real-time programs, input values usually come as strings.

** To perform calculations, we need to convert them into numbers.

Without Type casting

```
a = input("Enter first number : ")
b = input("Enter second number : ")
result = a + b #concatanation
print("Result Before Type Casting : ", result)
```

with type casting

```
a = int(a)
b = int(b)
result = a + b #Addition
print("Result After Type Casting : ", result)
```

list type casting

```
course_name = "Python"
letters = list(course_name)
print(letters)
```

Note: list() function converts every character of the string into a separate list element.

Tuple Type Casting

```
numbers = [10, 20, 30]
result = tuple(numbers)
print(result)
```

```
# Set Type Casting
numbers = [10, 20, 10, 30, 20]
unique_numbers = set(numbers)
print(unique_numbers)

# dict type casting
data = [("name", "Ashok"), ("course", "Python")]
student = dict(data)
print(student)

# Implicit casting
a = 10
b = 2.5
result = a + b
print("Result : ", result)
print(type(result))

# Explicit Casting
i = "10"
j = "20"
result = int(i) + int(j)
print("Result : ", result)
print(type(result))
```